

Conceptual Machine Engineering: Realization Contracts and Evidence-Governed Systems

Thomas Helm
Torafirma Systems Pty. Ltd.

8 October 2026
Methods manuscript, version 0.1

Abstract

A system description can determine the behavior required of several implementations while leaving their representations, resource costs, and operational permissions different. This paper presents Conceptual Machine Engineering (CME) as a contract-centered method for making those differences inspectable. An abstract transition system is paired with a realization contract specifying its validity domain, preserved observations, intentionally lost structure, failure semantics, error propagation, resource model, and evidence obligations. Conformance is distinguished from exact observational equivalence; tolerance-bounded agreement is not assumed to form equivalence classes. A separate admission model requires both evidence adequate for a particular candidate and authorization for the particular proposed state change. Elementary propositions establish finite-trace preservation, conditional error accumulation, and the absence of evidence-only state admission under explicit enforcement assumptions. A cyclic accumulator illustrates two representations, declared information loss, overflow and ordering counterexamples, and bounded executable checks. The contribution is an explicit methods synthesis and a reusable reporting discipline, rather than a new model of computation or a validated improvement in engineering performance. Its mathematical obligations reuse established specification, refinement, contract, and proof-carrying-code ideas. The method remains to be evaluated through independent reconstruction and comparative studies of realistic systems.

Keywords: realization contracts; formal specification; refinement; conformance; resource bounds; evidence provenance; authorization

1 Introduction

A common source of engineering error is a change in the meaning of a claim as it moves between abstraction levels. A mathematical invariant may become an assertion about finite-precision code. Agreement on final outputs may become a claim about execution traces. A passing test may become a reason to deploy a candidate for which no deployment decision was made. Each transition requires an argument, but the argument is often dispersed across a specification, implementation, test report, and operating procedure.

The practical question addressed here is narrower than the design of a universal engineering formalism: *what should be stated when several implementations are claimed to realize the same abstract machine, and what may legitimately follow from the evidence for that claim?* We use Conceptual Machine Engineering (CME) for a method that answers this question by making realization obligations and admission obligations explicit. The machine's specification

determines the selected semantics; the realization contract determines the scope of preservation; the evidence record supports a particular claim; and authorization determines whether a particular action may occur.

The term draws on the author’s earlier methodological work. *Research by Attractor* already develops motif discovery, cross-perspective examination, instrumentation, and evidence-sensitive promotion of ideas [6]. These are antecedents, not new findings of this paper. The October technical synthesis, advanced companion, and later consolidated Master Volume develop the realization vocabulary [5, 7, 8]. The present article narrows that material to self-contained contracts, finite-trace reasoning, and an illustrative executable case. It does not reproduce the earlier documents’ project inventory or treat documentary reports as independently verified implementation results.

There are three contributions. First, we specify a realization record whose semantic, resource, evidential, and action-permission obligations can be reviewed separately. Second, we state elementary sufficient conditions for composition and identify conditions that fail, including the nontransitivity of fixed-tolerance similarity. Third, we provide a minimal worked example and a conformance reporting protocol that can reject a candidate for a named reason. These contributions are methodological. No priority claim is made for abstraction-first development, contracts, refinement, or the distinction between semantic evidence and authority.

2 Position within established methods

Algebraic specification provides signatures, equations, and interpretations through which a data structure’s meaning can be separated from its representation. The initial-algebra work of Goguen and colleagues is a foundational account of this approach, including iterative and recursive semantic constructions [3]. CME reuses the idea that an implementation must preserve specified structure. It supplies no replacement semantics for algebraic theories.

Abstract state machines similarly support descriptions of algorithms at an appropriate abstraction level. Gurevich’s sequential ASM result characterizes a class of algorithms through explicit postulates and proves their representation by ASMs [4]. Our deterministic transition-system core is deliberately modest; it is not a competing characterization of algorithms. The question here is how to attach checkable realization, resource, and admission records to a chosen specification.

Refinement relates lower-level behavior to a higher-level specification. Abadi and Lamport study refinement mappings, including the use of auxiliary variables under conditions that guarantee their existence [1]. This is directly relevant when a concrete state contains bookkeeping absent from the abstract state. Section 4 uses a particularly simple lockstep abstraction map. Systems requiring stuttering, concurrency, or liveness arguments should use a suitable existing refinement theory rather than infer those properties from our one-step equations.

Model-driven engineering already puts models and transformations at the center of development, analysis, and artifact generation [10]. It also addresses multiple views and nonfunctional requirements. Accordingly, CME does not distinguish itself by asserting that model-driven engineering consists only of code generation. Contract-based design likewise has extensive formal accounts of component assumptions, guarantees, and composition [2]. A realization contract in this paper is an engineering record with explicit obligations; it is not claimed to be a new assume–guarantee calculus.

Proof-carrying code attaches a checkable safety argument to supplied code relative to a receiving system’s policy [9]. More closely still, Whitehead, Abadi, and Necula combine reasoning about program properties with authorization and trust in *By Reason and Authority* [11]. That work is important prior art for any proposal that connects evidence to permission.

Our two-predicate admission model is a simple reporting and enforcement pattern, not a new authorization logic. Its purpose is to expose which claim has been supported and which action has been permitted, including when the evidence is a test rather than a formal proof.

These connections define the contribution boundary. The synthesis is useful only if its explicit records help practitioners find missing obligations, invalid transfers, or stale evidence. Whether it improves that process over established methods is an empirical question, not a consequence of introducing the terminology.

3 Abstract machine and declared observations

Definition 1 (Machine used in this paper). *A deterministic conceptual machine is a tuple*

$$M = (\Sigma, X, X_0, U, T, Y, h, F),$$

where Σ is a typed vocabulary, X is a set of well-formed states, $X_0 \subseteq X$ is an initial-state set, U is the input set, $T : X \times U \rightarrow X$ is a partial transition function, $h : X \rightarrow Y$ is a declared observation function, and F is a set of typed failure outcomes. A contract specifies the meaning of undefined transitions and the behavior required when they are attempted.

The definition is a working scope, not a claim that all systems are deterministic or sequential. A nondeterministic machine requires a transition relation and a choice of trace inclusion, simulation, or another conformance relation. A physical realization additionally requires a measurement model linking observations to the formal quantities. Neither extension follows automatically from the deterministic case.

A finite input word $w = (u_0, \dots, u_{m-1})$ is admissible at x_0 when the successive states $x_{t+1} = T(x_t, u_t)$ all exist. Its observable trace is

$$\text{Obs}_M(x_0, w) = (h(x_0), h(x_1), \dots, h(x_m)).$$

The trace may include outcomes, event identities, and logical step numbers if those are contractually observable. Wall-clock time is included only through an explicit timing model. A statement about the last observation alone is weaker than a statement about the full trace.

Primitives and reduction. Every accepted derived construct should have a typed interpretation in the core vocabulary. A finite reduction tree is one way to demonstrate this. Alternatively, recursive constructs can be specified through an established semantic account with stated termination or fixed-point obligations. Finite syntactic decomposition does not by itself prove runtime termination, consistency, or usefulness. For rewrite-based definitions, termination without confluence can still leave several normal forms; the contract must resolve that ambiguity or admit it explicitly.

Invariants and observation loss. An invariant $I : X \rightarrow \{\text{true}, \text{false}\}$ is established by initialization and preservation: $I(x_0)$ for every admitted initial state and $I(x) \Rightarrow I(T(x, u))$ for every admitted step. An observation can discard information. If $h(x) = h(x')$ while $x \neq x'$, agreement of observations cannot establish equality of states. Whether that loss is acceptable depends on the property being claimed. For example, a final modular sum does not retain the order of increments or the number of completed cycles.

Table 1: Minimum realization contract. Each field names an obligation rather than supplying evidence merely by being present.

Field	Required statement
Identity and domain	Specification and candidate identities; initial states, inputs, horizon, environment assumptions, and excluded cases.
State and operator maps	Initialization, abstraction or simulation relation, concrete operators, and reachable-state conditions.
Preserved structure	Required observations, invariants, ordering, progress, and failure behavior.
Lost structure	Information erased by encoding, observation, batching, quantization, or an explicitly weaker contract.
Error model	Metric or relation, tolerances, local bounds, stability assumptions, and composition rule.
Resource model	Units, accounting boundary, execution schedule, bounds, and composition assumptions.
Evidence obligations	Claim type, checker or protocol, candidate binding, coverage, assumptions, and dependency records.
Action scope	Proposed operation and target; authority source and conditions; freshness and admission rules.

4 Realization contracts

A realization consists of concrete states Z , initialization e , concrete transitions R , and an abstraction map α defined on the admitted reachable states. The map $\alpha : Z \rightarrow X$ interprets a concrete state in the chosen abstract state space. An encoder $e : X_0 \rightarrow Z$ supplies initial representations and satisfies $\alpha(e(x_0)) = x_0$. The concrete state may contain caches, pipeline positions, logs, or other auxiliary information erased by α .

A realization contract K records the fields in Table 1. The specification identity, implementation identity, and evaluation identity are distinct. A versioned title is helpful for humans, but a receipt should bind the exact content evaluated. A hash establishes byte identity within its assumptions; it does not establish authenticity, correctness, or the truth of a reported result.

4.1 Exact semantic conformance

Exact lockstep conformance requires the following obligations. The first three apply to every contract-admitted reachable state z and valid input u . The fourth applies independently to every attempted operation whose rejection behavior is covered by K :

1. $\alpha(z)$ is defined and satisfies the abstract invariant;
2. if $(\alpha(z), u) \in \text{dom}(T)$, the concrete operation terminates successfully with a state $z' \in Z$ in the declared resource regime;
3. the preservation equation holds:

$$\alpha(z') = T(\alpha(z), u), \quad z' = R(z, u); \quad (1)$$

4. if the contract requires rejection because an input or state is outside the permitted domain, the operation terminates with the specified failure outcome and specified state effect, including no state change when rejection is atomic. Subsequent requests follow the same declared failure and recovery semantics.

Here $R(z, u)$ denotes the state component of a successful operation; failure outputs are handled by the fourth obligation. Equation (1) is checked on *all admitted reachable states*, not only on freshly encoded states. An implementation can leave the image of e after one step without violating conformance, but subsequent steps still require justification. Conversely, testing only $R(e(x), u)$ can miss errors in auxiliary state accumulated over time.

Proposition 1 (Finite-trace preservation). *Suppose initialization and the four exact obligations hold throughout an admissible word w . Then the decoded concrete states coincide with the abstract states at each logical step, and their h -observations coincide for the entire word.*

Proof. Initialization gives $\alpha(z_0) = x_0$. If $\alpha(z_t) = x_t$, successful termination and equation (1) give $\alpha(z_{t+1}) = T(x_t, u_t) = x_{t+1}$. Induction over the finite length of w proves the state claim; applying h proves the observation claim. $\square$

This elementary argument is not a general refinement-completeness theorem. It excludes unmodeled concurrency, hidden input changes, and infinite-run liveness. A concrete implementation that waits forever while leaving its state correct would violate the successful-termination obligation even though no wrong final value is produced.

4.2 Approximate conformance and composition

Let the retained abstract state carry a metric d . Assume the abstraction of each realized successor is within δ_t of applying the abstract transition to the currently decoded state:

$$d(\alpha(R(z_t, u_t)), T(\alpha(z_t), u_t)) \leq \delta_t.$$

Both $T(x_t, u_t)$ and $T(\alpha(z_t), u_t)$ must be defined in the declared domain. Suppose also that $T(\cdot, u_t)$ is L_t -Lipschitz there.

Proposition 2 (Conditional accumulated error). *Writing $e_t = d(\alpha(z_t), x_t)$, one has*

$$e_{t+1} \leq L_t e_t + \delta_t,$$

and consequently

$$e_m \leq e_0 \prod_{j=0}^{m-1} L_j + \sum_{i=0}^{m-1} \delta_i \prod_{j=i+1}^{m-1} L_j. \quad (2)$$

Empty products equal one. The bound applies only while all domain and stability assumptions hold.

Proof. Insert $T(\alpha(z_t), u_t)$ between $\alpha(z_{t+1})$ and $T(x_t, u_t)$ in the triangle inequality. The local-error assumption bounds the first term by δ_t ; the Lipschitz assumption bounds the second by $L_t e_t$. Iterating this recurrence proves (2). $\square$

A blanket rule that errors always add is therefore unjustified. Addition follows when $L_t \leq 1$ and the same metric and domain apply. With constant $L > 1$, local error can be amplified. If observations have a Lipschitz constant L_h , their error is at most $L_h e_t$. Discontinuous observations, such as a threshold-triggered authorization decision, require a separate robustness margin or exact checking. A small numeric residual cannot silently relax an exact action boundary.

4.3 Equivalence, conformance, and allowed loss

Definition 2 (Exact observational equivalence). *On a fixed common input domain, initial-state interface, and observation contract K , two terminating realizations are observationally equivalent when their complete decoded observable traces and declared failure outcomes agree for every input word covered by K .*

Equality of trace functions is reflexive, symmetric, and transitive. It therefore supports equivalence classes. Two exact realizations agree on admitted traces by Proposition 1; when their failure and recovery rules also agree, extending the stepwise argument to rejected requests gives equivalence on the full contract. Neither statement equates their source code, internal states, resource costs, or security properties outside the observation contract.

By contrast, fixed-tolerance closeness is generally nontransitive. For $\varepsilon > 0$, the real values 0, 0.75ε , and 1.5ε have adjacent distances below ε , while their endpoint distance exceeds ε . Two realizations each within ε of one reference can differ by as much as 2ε . We therefore call them *separately conformant to a reference at a stated tolerance*; we do not form a quotient by an unproved approximate-equivalence relation.

A weakened observation contract must be explicit. Discarding intermediate outputs can make reordered operations agree at the endpoint while violating a trace contract. If a realization cannot retain the specified abstract state, it needs a weaker abstract machine or an appropriate simulation relation with proved properties. Listing a required invariant under “lost structure” does not discharge its preservation obligation.

5 Resource bounds and validity regimes

Resource assertions require their own units, boundaries, and composition rules. A useful record might contain total work W , retained memory M , communication volume C , and a time bound T_{wall} . These quantities are not interchangeable. Counting one abstract transition as one unit does not imply constant physical cost, and equal behavior does not imply equal cost.

For sequential stages, elapsed upper bounds can be added only when each bound applies to the actual execution regime and the accounting includes transitions between stages. For independently scheduled parallel stages, a maximum may bound an idealized critical path, but contention, synchronization, and queues require additional terms. Peak memory is the maximum amount live at any instant. Reusing a scratch buffer can yield $B + \max_i S_i$ for persistent memory B and stage scratch sizes S_i ; retaining outputs between stages invalidates that formula. A universally safe upper bound may be less informative than a schedule-sensitive analysis.

For a fixed sequence of failure events E_i with valid individual probability bounds $\Pr(E_i) \leq p_i$, the union bound gives

$$\Pr\left(\bigcup_i E_i\right) \leq \min\left(1, \sum_i p_i\right).$$

No independence assumption is needed for this inequality. Multiplicative survival calculations do require appropriate probabilistic assumptions. Estimated probabilities also carry statistical uncertainty; merely putting them in a resource vector does not turn estimates into guarantees.

A *validity regime* is the set of conditions for which the realization’s stated obligations apply. It can depend on input shape, numerical range, precision, workload distribution, timing, hardware, or failure model. The specification domain, proven domain, and tested points should be listed separately. A finite test set does not establish an entire interpolated region. When

no applicable evidence is available, the disposition is *unknown*; a proved impossibility, an observed failure, and an economically unattractive option are different conclusions.

When a controller switches between individually conformant realizations, the transfer of state and the switching rule are additional operators. Their buffering, ordering, cost, and error obligations must be checked. Correctness of two kernels does not establish correctness of their composite switching system.

6 Evidence records and action authority

An evidence record E should identify a claim q , the candidate and specification versions, the evaluation domain, method, result, assumptions, and dependencies. Useful evidence classes include a deductive argument, a mechanically checked proof, a bounded exhaustive check, sampled tests, and measurements with a specified uncertainty model. These classes answer different questions; they need not form a single linear confidence ladder.

A proof about an abstract transition and a sample of implementation tests jointly support an abstract theorem and sampled conformance. They do not establish universal implementation correctness. Likewise, independent-looking reports may share a parser, oracle, dataset, or upstream argument. A dependency record helps distinguish repeated evidence from methodologically independent evidence. Statistical combination requires an explicit dependence model; there is no universal product rule for “confidence.”

6.1 Candidate-bound admission

Consider an admission service with state (c, v, L) : admitted state c and version v of the named target, and an evidence log L . A request

$$r = (i_M, i_R, a, \tau, v, c')$$

names a specification identity i_M , candidate identity i_R , action a including its parameters, target τ , required current version v , and proposed next state c' . The evidence-adequacy predicate $\text{Adeq}(E, r)$ means that the evidence meets the declared policy for this exact request. The authorization predicate $\text{Auth}(P, r, s)$ means that permission record P authorizes the same request under the current authority state s .

A simple fail-closed admission rule is

$$\text{Accept}(r) = \text{Adeq}(E, r) \wedge \text{Auth}(P, r, s) \wedge \text{Fresh}(r, c, v) \wedge \text{Valid}(r). \quad (3)$$

The last predicate includes the target’s invariant and any remaining transition constraints. An undecided or missing predicate prevents commitment. The atomic commit updates c and v only when (3) holds. Rejection may append a result to L , but leaves the admitted state unchanged.

Proposition 3 (No evidence-only admission). *Assume every write to the admitted target is mediated by rule (3), the request is immutable, the rule and authority state cannot be changed by submitting evidence, and all admission predicates are evaluated against the same consistent target and authority state at the atomic commit point. If $\text{Auth}(P, r, s)$ is false at that point, submitting or appending evidence for r cannot commit its proposed target-state change.*

Proof. The false authorization conjunct makes $\text{Accept}(r)$ false regardless of the evidence value. By the stipulated transition rule, a rejected request leaves the target state unchanged. Since every write is mediated by that rule and evidence cannot change it, no evidence-only submission supplies an alternative write path. $\square$

The assumptions are the substance of the security obligation. The proposition is a property of the specified service, not a verified claim about a deployed system. Implementation checks must address bypass paths, authentication, permission expiry and revocation, target binding, atomicity, and changes between checking and use. An append-only log aids traceability only within its storage and access assumptions; append-only structure alone proves neither authenticity nor truth.

The separation also works in the other direction: authorization does not establish correctness. A person can permit a bounded test whose outcome is unknown. An evidence threshold can permit a candidate to be considered for release without itself authorizing release. Whether a policy deliberately treats particular evidence as sufficient grounds for an automated action is a separate, explicit policy decision with a defined scope.

7 Worked example: a bounded cyclic accumulator

The source documents use a cyclic accumulator to illustrate realization [5, 7]. We adopt a smaller, fully specified variant. It preserves residue and logical step count, while deliberately omitting wrap count and prior input history. The example is chosen for inspectability, not as a difficult algorithmic result.

7.1 Abstract contract

Freeze $N \geq 2$ and a finite horizon $H \geq 0$. Let

$$X = \{0, \dots, N-1\} \times \{0, \dots, H\}, \quad U = \{0, \dots, N-1\},$$

with initial states $(x, 0)$ and identity observation $h(x, k) = (x, k)$. For $k < H$ define

$$T((x, k), u) = ((x + u) \bmod N, k + 1). \quad (4)$$

An input outside U , an ill-formed state, or a step attempted at $k = H$ produces a domain failure without changing state. The transcript contract observes each accepted state, so input order matters. This example assumes sequential, failure-free execution of accepted operations; it makes no crash-recovery promise.

For a length- m admitted input word,

$$x_m = \left(x_0 + \sum_{t=0}^{m-1} u_t \right) \bmod N, \quad k_m = m. \quad (5)$$

This follows by induction using modular addition. Range and count bounds are invariants. The current state generally does not recover the inputs or total wraps. For $N = 5$, words $(1, 2)$ and $(4, 4)$ from zero both end at state $(3, 2)$, while their integer totals and wrap counts differ. If accounting for cumulative integer value were required, an additional state coordinate and contract would be necessary.

7.2 Two representations

The register realization stores (x, k) directly and performs integer addition followed by reduction modulo N . A rotating-marker realization stores (b, k) , where $b = 2^x$ is an N -bit word with exactly one bit set. Its abstraction is $\alpha(b, k) = (\log_2 b, k)$ on these one-hot states. For an admitted u , it rotates the marker u places modulo N and increments k :

$$R_{\text{ring}}((b, k), u) = \left(((b \ll u) \text{ or } (b \gg (N - u))) \text{ and } (2^N - 1), k + 1 \right).$$

Table 2: Explicit counterexamples and their contractual significance. Arithmetic values are exact.

Case	Result	Missing obligation
Overflow before reduction	For $N = 251$, $x = 250$, $u = 10$: exact residue 9; eight-bit wrap followed by reduction gives 4.	Width or overflow-safe arithmetic.
Input reordering	From zero modulo 5: $(1, 2)$ gives residues $(1, 3)$; $(2, 1)$ gives $(2, 3)$.	Order-sensitive trace preservation.
Duplicate delivery	$(1, 2)$ ends at 3 modulo 5; $(1, 2, 2)$ ends at 0.	Delivery multiplicity or explicit deduplication.
Loss of wrap history	$(1, 2)$ and $(4, 4)$ both end at $(3, 2)$ modulo 5, but have different integer totals.	A retained wrap or cumulative-total coordinate.
Evidence without permission	Candidate checks pass; authorization for the named target is absent.	A separately satisfied authorization predicate.

The operators denote unsigned shifts and bitwise operations on nonnegative integers. For $u = 0$, the right-shifted term is zero on admitted states and the marker is unchanged. For all admitted u , the set bit moves from position x to $(x + u) \bmod N$. Thus both realizations satisfy (1); Proposition 1 supplies finite-trace preservation.

The representations have different logical storage requirements: $\lceil \log_2 N \rceil$ bits for the residue in a packed register and N bits for a one-hot word, plus $\lceil \log_2(H + 1) \rceil$ count bits in either representation. These are representation sizes, not measured Python memory footprints. A hardware barrel shifter, a multiword software shift, and a register addition have different cost models. The demonstration therefore makes no speed comparison and does not claim that one semantic step is one machine instruction.

7.3 Counterexamples that distinguish contracts

Table 2 makes the weaker claims visible. A machine can preserve modular range while computing the wrong residue; it can preserve the final residue while changing the trace; and it can produce a mathematically valid candidate without permission to admit it.

For a w -bit unsigned intermediate, $x + u \leq 2N - 2$ in this domain. A direct nonoverflowing addition is guaranteed when $2N - 2 \leq 2^w - 1$, followed by correct modular reduction. Wider intermediates or a proved overflow-safe algorithm can support other ranges. Overflow behavior must not be inferred from the algebra of residues alone.

7.4 Executable illustration and exact scope

The supplement contains a small Python implementation, a cumulative-sum oracle, the two representations, and a toy admission rule. Executed with CPython 3.12.14, it checked all 1,784 triples (N, x, u) with $2 \leq N \leq 17$ and $0 \leq x, u < N$ for one-step agreement. It also checked all 3,905 initial-state/input-word cases for $N = 5$ and lengths 0 through 4, comparing every prefix and the cumulative-sum oracle. Eighteen counterexample and rejection checks and one authorized-admission positive control produced their specified outcomes. The source digest and exact check definitions are recorded with the result.

These counts describe finite, reproducible checks of this illustration. They do not establish throughput, production security, hardware behavior, or the effectiveness of CME as a development method. Both realizations and the checking code were prepared together; they are not independent reconstructions. The proofs above establish the stated mathematical identities,

while the executable checks provide bounded evidence about the accompanying code. No proof assistant was used. In the toy admission rule, permission and evidence matching are data comparisons. Its validity predicate checks the fixed accumulator transition, action, target, and specification identity. Authentication and atomic concurrent storage are assumed, not implemented.

8 Conformance and reporting protocol

A useful application begins by freezing a claim rather than selecting a favorable implementation result. The following protocol is intended for a small reproducibility package or an engineering review.

1. **State the observable claim.** Identify whether it concerns full state, a trace, endpoint behavior, a probability distribution, resource use, or a particular action boundary. Declare the domain and excluded failure conditions.
2. **Freeze identities and assumptions.** Record specification, source, build, configuration, input data, evaluator, and measurement protocol. Any changed dependency requires a decision about which claims remain valid.
3. **Discharge structural obligations.** Review types, initialization, reachable-state invariants, abstraction maps, allowed loss, operator preservation, and successful termination. Record which are proved and which are only tested.
4. **Test boundaries and counterexamples.** Include valid extremes, rejected inputs, overflow or precision boundaries, ordering changes, duplicate events, stale evidence, and unauthorized actions where applicable. Specify expected outcomes before evaluation.
5. **Evaluate composition.** Check domains between stages, accumulated error, resource accounting, state transfer, and assumptions shared by evidence producers. A local pass does not fill a missing composition argument.
6. **Record a scoped disposition.** Report pass, fail, unknown, or inapplicable for each claim, with evidence type and coverage. Preserve the failed and blocked cases. Evaluate action permission separately for any proposed commitment.

The conformance corpus should expose enough information for a new implementation to fail unambiguously. Reference outputs are useful, but they are not a complete specification. Metamorphic relations and independently derived properties can reveal a shared error in two implementations and their common oracle. The independent-reconstruction question is whether an implementer can recover the declared behavior from the specification and public conformance surface. Re-executing the same program instead addresses artifact reproducibility. Neither experiment dominates the other for every scientific purpose.

A coverage statement should name its measure. “All inputs” is justified for an exhaustively enumerated finite domain, not for an unbounded domain sampled at several points. Coverage of a parameter region requires a stated measure or count and evidence for inclusion; a colored plot alone supplies neither. A disagreement should be localized to an assumption, abstraction, operator, observation, or evaluator before it is attributed to a novel mechanism.

9 Limitations and evaluation agenda

The framework organizes obligations; it does not manufacture their evidence. A complete record can contain false statements, a checker can implement the wrong policy, and a proof

can concern a model that fails to represent the intended system. The trust boundary therefore includes the specification, maps, evaluators, measurement procedures, storage, and authority mechanism. More elaborate forms do not remove these dependencies.

The formal propositions deliberately cover simple conditions. They establish no automatic synthesis procedure, completeness result for arbitrary refinement, universal evidence algebra, optimal resource allocation, or guarantee for physical realization. The accumulator is too small to assess specification burden, multi-team coordination, and realistic failure recovery. The bounded checks are a worked artifact, not empirical validation of the general method. Heterogeneous implementations may expose more failure modes, but shared assumptions can still produce shared errors.

A comparative evaluation should examine whether this explicit record improves a defined task over an appropriate baseline, such as an ordinary specification plus tests or an established contract/refinement workflow. Useful outcomes include omitted-obligation detection, incorrect acceptance rate, effort to produce a second realization, ambiguity-resolution time, and engineering effort required to maintain the records. Tasks, acceptance criteria, evaluator independence, and stopping rules should be fixed before outcomes are inspected. No such comparative study is reported here.

Independent reconstruction should be evaluated separately from algorithmic diversity. A second implementation authored by the same team can be valuable differential evidence without satisfying an independence claim. A reconstruction study should document what sources the second team could see, how specification questions were answered, and which revisions were required. Any disclosure restriction limits the claims that another party can audit; specification sufficiency does not establish the correctness or performance of an undisclosed optimized implementation.

Finally, the synthesis remains open to a strong null interpretation: established specification, refinement, contract, and authorization methods may already provide all of its formal content. Its research value would then lie in demonstrably useful integration and reporting. A broader theory or novelty claim would require a distinct technical result and a more comprehensive literature review.

10 Conclusion

Realization claims are meaningful only relative to declared structure, observations, and operating conditions. CME packages these obligations into a contract that records what is preserved, what is lost, how error and resources are accounted for, and what evidence supports the particular candidate. Exact observational equivalence and bounded conformance have different logical properties. Evidence for either remains distinct from authorization to enact a state change.

The resulting method is intentionally testable at its interfaces. A realization can fail a state map, trace requirement, error bound, resource assumption, or admission rule without the failure being hidden in one aggregate success label. The next substantive step is to evaluate that discipline on independent realizations and realistic engineering tasks. The present paper supplies the definitions, conditional arguments, and small reproducible example needed to make that evaluation precise.

Data and code availability

The accompanying supplement contains `check_example.py`, its machine-readable results, and reproduction instructions. It uses exact Python integer arithmetic and no third-party packages. The manuscript is self-contained with respect to its definitions and proofs. The unpublished precursor manuscripts are cited for intellectual provenance, not as substitutes for inspectable evidence. This version includes no empirical claims about their underlying systems.

Version and provenance

This methods manuscript is a scoped development of the author’s October 2026 CME materials and acknowledges the prior public *Research by Attractor* work. The source chronology is recorded in the accompanying source and claim ledger. The present version corrects the use of tolerance-bounded similarity as an equivalence relation, makes error-composition assumptions explicit, and limits the accumulator contract to the information it actually retains.

A Compact realization and admission record

For reuse, a record can contain the following fields. A machine-readable serialization should preserve the separation between claim, evidence, and requested action; the field names alone carry no trust.

Claim

Specification identity; candidate identity; required observations; validity domain; horizon; invariants; excluded conditions.

Realization

Initial encoding; abstraction map or simulation relation; concrete operator bindings; retained and lost structure; progress and failure semantics.

Bounds

Metric and tolerance; local and composed error arguments; resource units and accounting boundary; schedule and environment assumptions.

Evidence

Claim identifier; method and evidence class; source/build/data/evaluator identities; exact coverage; result; unresolved assumptions; shared dependencies; superseded records.

Admission

Exact proposed action, target, candidate and before-state identity; evidence-policy result; separately bound permission; freshness check; atomic commit or rejection outcome.

A receipt should be interpretable without reconstructing the author’s working environment. Public supplements should contain only the provenance needed to understand and reproduce the published claim.

References

- [1] Martín Abadi and Leslie Lamport. The existence of refinement mappings. *Theoretical Computer Science*, 82(2):253–284, 1991. doi: 10.1016/0304-3975(91)90224-P. URL <https://lamport.azurewebsites.net/pubs/abadi-existence.pdf>.

- [2] Albert Benveniste, Benoît Caillaud, Dejan Nickovic, Roberto Passerone, Jean-Baptiste Raclet, Philipp Reinkemeier, Alberto Sangiovanni-Vincentelli, Werner Damm, Thomas A. Henzinger, and Kim G. Larsen. Contracts for system design. *Foundations and Trends in Electronic Design Automation*, 12(2–3):124–400, 2018. doi: 10.1561/10000000053. URL <https://www.nowpublishers.com/article/DownloadSummary/EDA-053>.
- [3] Joseph A. Goguen, James W. Thatcher, Eric G. Wagner, and Jesse B. Wright. Initial algebra semantics and continuous algebras. *Journal of the ACM*, 24(1):68–95, 1977. doi: 10.1145/321992.321997. URL <https://research.ibm.com/publications/initial-algebra-semantics-and-continuous-algebras>.
- [4] Yuri Gurevich. Sequential abstract-state machines capture sequential algorithms. *ACM Transactions on Computational Logic*, 1(1):77–111, 2000. doi: 10.1145/343369.343384. URL <https://web.eecs.umich.edu/~gurevich/Opera/141.pdf>.
- [5] Thomas Helm. Conceptual machine engineering: Advanced techniques. Methodology companion, version 0.1, 4 October 2026. Unpublished manuscript, 2026.
- [6] Thomas Helm. Research by attractor: Motif gravity, perspectival orbiting, and the instrumented promotion of cognitive operators, July 2026. URL https://www.academia.edu/169933252/Research_by_Attractor_Motif_Gravity_and_Perspectival_Orbiting. Version 1.0, 9 July 2026. Published under the name Umbra Fidelis.
- [7] Thomas Helm. Conceptual machine engineering: Master volume. Version 1.0, 4 October 2026. Unpublished manuscript, 2026.
- [8] Thomas Helm. Conceptual machine engineering: Constructing executable systems in abstract space before implementation. Technical synthesis, version 0.1, 4 October 2026. Unpublished manuscript, 2026.
- [9] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 106–119. ACM, 1997. doi: 10.1145/263699.263712. URL <https://www.cs.tufts.edu/comp/150CMP/papers/necula97pcc.pdf>.
- [10] Douglas C. Schmidt. Guest editor’s introduction: Model-driven engineering. *Computer*, 39(2):25–31, 2006. doi: 10.1109/MC.2006.58. URL <https://www.dre.vanderbilt.edu/~schmidt/PDF/GEI.pdf>.
- [11] Nathan Whitehead, Martín Abadi, and George C. Necula. By reason and authority: A system for authorization of proof-carrying code. In *Proceedings of the 17th IEEE Computer Security Foundations Workshop*, pages 236–250. IEEE, 2004. doi: 10.1109/CSFW.2004.6. URL https://people.eecs.berkeley.edu/~necula/Papers/sigpcc_csfw04.pdf.